

Philipp Hilbert. *Enterprise AI product, shipped under sovereign constraint.*

Positioning, ten years building regulated-industry platforms, now applying that to agentic AI that has to work inside the security perimeter, not around it.

BERLIN, GERMANY · HILBERT@TRUE-NORTH.BERLIN · [TRUE-NORTH.BERLIN](https://www.true-north.berlin)

[LinkedIn](#) · [GitHub](#) · www.true-north.berlin · [YouTube \(EN sample\)](#)

Dear ai71 team,

I am applying for the Product Manager role because the problem you are solving is the one I find most interesting in enterprise AI right now: making agentic systems trustworthy enough to run a ministry's legal review or a bank's procurement workflow, inside the customer's own perimeter, without any of the escape hatches that make demos look better than they are.

My most relevant experience for this role is at Rohde & Schwarz, where I owned the defence AI platform for the FCAS programme. The environment there was not air-gapped, but it carried the same procurement weight, classification sensitivity, and expectation of auditability that you describe. I moved that platform from a UI-heavy approach to a developer-code-friendly one, cut rollout time from days to thirty minutes, and made the system fully hotfix-capable, all without the freedom to iterate after go-live that a consumer product would allow. Before that, at Bundesdruckerei, I owned the national health data platform for the RKI, including the vaccination statistics and viral-mutation tracking systems that ran under Germany's highest privacy requirements. Both roles required treating compliance and auditability as first-class product requirements, not layers bolted on at the end.

The eval ownership requirement in your job description is the piece I most want to address directly, because it is the one most PM roles avoid. At construct8, I own the quality bar for LLM-agent outputs end-to-end, defining what "correct" means, building test sets, and making the agent's behaviour measurable before the customer sees it. I built my own agent framework as a side project, including a backtesting engine that functions as a ground-truth judge before any strategy runs on real capital. The pattern is the same: you cannot ship a probabilistic system into a regulated environment and rely on "it looked fine in the demo." I share that view, and I know how to build the scaffolding that makes quality visible.

On the builder requirement: I prototype in Lovable, run multi-agent workflows in n8n, and ship feature-complete code via Claude Code and GitHub Copilot. A disputed flow becomes a working artefact the same day. I bring that to product debates rather than a slide deck, and I expect that to be useful in a squad that ships and learns faster than the roadmap predicts.

The forward-deployed track is where I would want to start. The shortest distance between a product decision and a real operation is exactly the feedback loop that makes the rest of the roadmap honest.

Best regards, *Philipp Hilbert*
